

ECE391
Computer System Engineering
Lecture 12

Prof. Kirill Levchenko and Dong Kai Wang

Slides adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

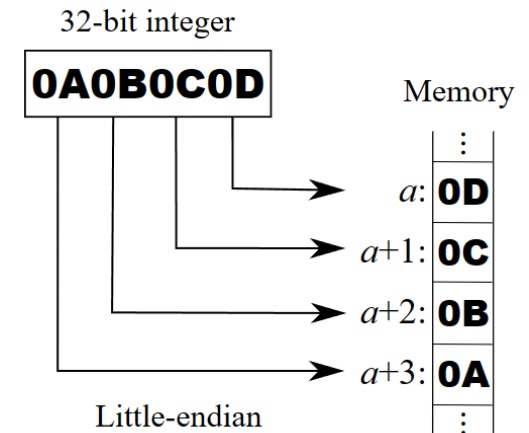
Spring 2024

Announcements

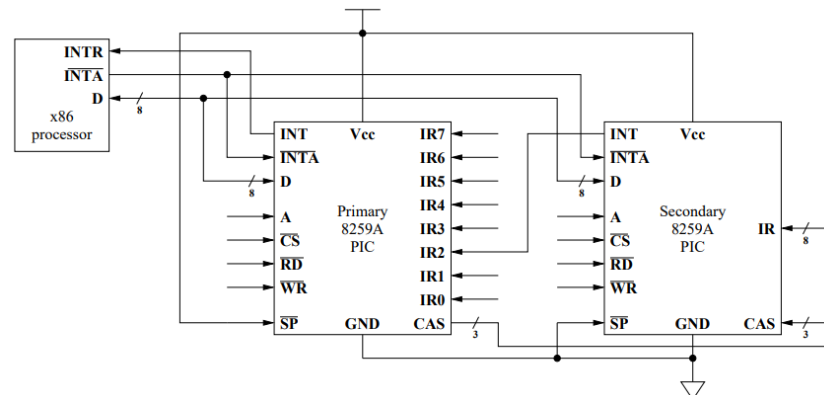


- Exams will be handed back at the end of lecture.
 - No individual curve on exams, curve at the end of semester.
- Regrade requests due by next Tuesday 5PM.
- MP2 deadlines are approaching.
- Read ahead in CN to pg. 55, finish ULK4.

- In x86, why do we prefer memory addresses to be multiples of 4?



- How does the 8259A PIC know whether it is a master or slave?
-
- The diagram shows a block labeled 'INTR' with an arrow pointing to it from a horizontal line. This horizontal line is connected to a vertical line that has a terminal symbol (a small circle with a cross) at the top. This represents the master/slave selection logic where the INTR pin is connected to a common bus and a specific terminal to determine its role.



Lecture Topics

- Recap: PIC Initialization
- Interrupt Jump Tables
 - Key Functions
- Interrupt Chaining

Recall: 8259A Commands

- 8259A accepts two types of command words:
 - Initialization **C**ommand **W**ords (**ICW**)
 - Operation **C**ommand **W**ords (**OCW**)
- Use the x86 in/out instructions to write these words.
 - Any command with A = 0 (writing to 0x20) and D4 = 1 is recognized as ICW1.
 - An initial write to A = 1 (writing to 0x21), after initialization, modifies masks registers.

Operation Control Words (OCWs)

OCW1								
A0	D7	D6	D5	D4	D3	D2	D1	D0
1	M7	M6	M5	M4	M3	M2	M1	M0

- Linux source code:

<https://elixir.bootlin.com/linux/2.4.28/source/arch/i386/kernel/i8259.c>

Code specific to
x86 architecture

Linux 8259A Initialization

```
void init_8259A(int auto_eoi)
{
    unsigned long flags;

    i8259A_auto_eoi = auto_eoi;
    spin_lock_irqsave(&i8259A_lock, flags);

    outb(0xff, 0x21);      /* mask all of 8259A-1 */
    outb(0xff, 0xA1);      /* mask all of 8259A-2 */

    /*
     * outb_p - this has to work on a wide range of PC hardware.
     */
    outb_p(0x11, 0x20);    /* ICW1: select 8259A-1 init */
    outb_p(0x20 + 0, 0x21); /* ICW2: 8259A-1 IR0-7 mapped to 0x20-0x27 */
    outb_p(0x04, 0x21);    /* 8259A-1 (the primary) has a secondary on IRQ2
    if (auto_eoi)
        outb_p(0x03, 0x21);
    else
        outb_p(0x01, 0x21);

    outb_p(0x11, 0xA0);    /* ICW1: select 8259A-2 init */
    outb_p(0x20 + 8, 0xA1); /* ICW2: 8259A-2 IR0-7 mapped to 0x28-0x2f */
    outb_p(0x02, 0xA1);    /* 8259A-2 is a secondary on primary's IRQ2
    outb_p(0x01, 0xA1);

    if (auto_eoi)
        /*
         * in AEOI mode we just have to mask the interrupt
         * when acking.
         */
        i8259A_irq_type.ack = disable_8259A_irq;
    else
        i8259A_irq_type.ack = mask_and_ack_8259A;

    udelay(100);          /* wait for 8259A to initialize */

    outb(cached_21, 0x21);
    outb(cached_A1, 0xA1);

    spin_unlock_irqrestore(&i8259A_lock, flags);
}
```

Comments on Linux' 8259A Initialization Code

1. What is the `auto_eoi` parameter? **always = 0 in Linux**
2. Four initialization control words to set up primary 8259A.
3. Four initialization control words to set up secondary 8259A.

	port(A=?)	info contained in Initialization Control Word
ICW1	0	start init, edge-triggered inputs, cascade mode, 4 ICWs
ICW2	1	high bits of vector #
ICW3	1	primary PIC: bit vector of secondary PIC; secondary PIC: input pin on primary PIC
ICW4	1	ISA=x86, normal/auto EOI

Comments on Linux' 8259A Initialization Code (cont.)

5. What does the “**_p**” mean on the “outb” macros?
 - add PAUSE instruction after OUTB; “REP NOP” prior to P4.
 - delay needed for old devices that cannot handle processor's output rate.
6. Critical section spans the whole function.
 - avoid other 8259A interactions during initialization sequence.
 - (device protocol requires that four words be sent in order).
7. Why **use _irqsave** for critical section? (*difference with _irq?*)
 - may be called other interrupt initialization routines. “re-initialized”
 - unknown state of IF flag.

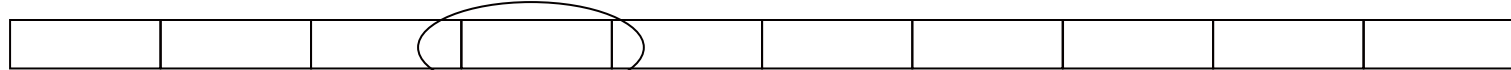
```
apic_write_around(  
init_8259A(1);  
timer_ack = 1;
```

Interrupt Descriptor Table

0x00–0x1F defined by Intel	0x00	division error	
	⋮		
	0x02	NMI (non-maskable interrupt)	
	0x03	breakpoint (used by KGDB)	
	0x04	overflow	
	⋮		
	0x0B	segment not present	
	0x0C	stack segment fault	
	0x0D	general protection fault	
	0x0E	page fault	
	⋮		
0x20–0x27 master 8259 PIC	0x20	IRQ0 — timer chip	example of possible settings
	0x21	IRQ1 — keyboard	
	0x22	IRQ2 — (cascade to slave)	
	0x23	IRQ3	
	0x24	IRQ4 — serial port (KGDB)	
	0x25	IRQ5	
	0x26	IRQ6	
	0x27	IRQ7	
0x28–0x2F slave 8259 PIC	0x28	IRQ8 — real time clock	
	0x29	IRQ9	
	0x2A	IRQ10	
	0x2B	IRQ11 — eth0 (network)	
	0x2C	IRQ12 — PS/2 mouse	
	0x2D	IRQ13	
	0x2E	IRQ14 — ide0 (hard drive)	
	0x2F	IRQ15	
0x30–0x7F	⋮	APIC vectors available to device drivers	
0x80	0x80	system call vector (INT 0x80)	
0x81–0xEE	⋮	more APIC vectors available to device drivers	
0xEF	0xEF	local APIC timer	
0xF0–0xFF	⋮	symmetric multiprocessor (SMP) communication vectors	

Linux' Interrupt Data Structures

IRQ descriptor array (irq_desc)



status
chip
action list
disable depth
lock
flow handler

**PIC
jump
table**

Focus of Today

Source Code: irq_desc

- [irq.h - include/linux/irq.h - Linux source code \(v2.6.21\) - Bootlin](#)

```
struct irq_desc {
    irq_flow_handler_t    handle_irq;
    struct irq_chip        *chip;
    struct msi_desc        *msi_desc;
    void                  *handler_data;
    void                  *chip_data;
    struct irqaction        *action;    /* IRQ action list */
    unsigned int           status;      /* IRQ status */

    unsigned int           depth;        /* nested irq disables */
    unsigned int           wake_depth;   /* nested wake enables */
    unsigned int           irq_count;    /* For detecting broken IRQs */
    unsigned int           irqs_unhandled;
    spinlock_t             lock;

#ifdef CONFIG_SMP
    cpumask_t              affinity;
    unsigned int           cpu;
#endif

#ifdef CONFIG_GENERIC_PENDING_IRQ || defined(CONFIG_IRQBALANCE)
    cpumask_t              pending_mask;
#endif

#ifdef CONFIG_PROC_FS
    struct proc_dir_entry  *dir;
#endif

    const char             *name;
} ____cacheline_aligned;
```

Linux Abstraction of PICs

- Uses a jump table:
 - same as vector table (array of **function pointers**).
 - generic functions needed by all.
- Table is *hw_irq_controller* structure (or struct irq_chip)
 - each vector # associated with a table.
 - table used to interact with appropriate PIC (e.g., 8259A, or Advanced PIC).

human-readable name
startup function
shutdown function
enable function
disable function
mask function
mask_ack function
unmask function
(+ several others...)

Linux Abstraction of PICs

- *hw_irq_controller* structure definition:
 - IRQs are #’d 0-15 (correspond to vector # - 0x20)

```
const char* name;  
unsigned int (*startup)(unsigned int irq);  
void (*shutdown)(unsigned int irq);  
void (*enable)...  
void (*disable)...  
void (*ack)...  
void (*end)...  
/* we'll ignore the others... */
```



8259A's human-readable name: "XT-PIC" (why?); see /proc/interrupts

Source Code

- [irq.h - include/linux/irq.h - Linux source code \(2.4.28\) – Bootlin](#)

```
/*
 * Interrupt controller descriptor. This is all we need
 * to describe about the low-level hardware.
 */
struct hw_interrupt_type {
    const char * typename;
    unsigned int (*startup)(unsigned int irq);
    void (*shutdown)(unsigned int irq);
    void (*enable)(unsigned int irq);
    void (*disable)(unsigned int irq);
    void (*ack)(unsigned int irq);
    void (*end)(unsigned int irq);
    void (*set_affinity)(unsigned int irq, unsigned long mask);
};

typedef struct hw_interrupt_type hw_irq_controller;
```

Address to call {

Example

- Example default with dummy functions.

```
struct hw_interrupt_type no_irq_type = {  
    "none",  
    startup_none,  
    shutdown_none,  
    enable_none,  
    disable_none,   
    ack_none,  
    end_none  
};  
  
/*  
 * Generic no controller code  
 */  
  
static void enable_none(unsigned int irq) { }  
static unsigned int startup_none(unsigned int irq) { return 0; }  
static void disable_none(unsigned int irq) { }  
static void ack_none(unsigned int irq)
```

Hint: don't need & here (C assumes this is func. address)



Jump Table for 8259A PIC

- <https://elixir.bootlin.com/linux/2.4.28/source/arch/i386/kernel/i8259.c#L151>

```
static struct hw_interrupt_type i8259A_irq_type = {  
    "XT-PIC",  
    startup_8259A_irq,  
    shutdown_8259A_irq,  
    enable_8259A_irq,  
    disable_8259A_irq,  
    mask_and_ack_8259A,  
    end_8259A_irq,  
    NULL  
};
```

- Corresponding functions for each IRQ.
- Not for the whole PIC.

Auto EOI Function Example

- Recall setting the auto-EOI mode in `init_8259A`.
 - Set different function pointers depending on mode.

```
if (auto_eoi)
    /*
     * in AEOI mode we just have to mask the interrupt
     * when acking.
     */
    i8259A_irq_type.ack = disable_8259A_irq;
else
    i8259A_irq_type.ack = mask_and_ack_8259A;
```

PIC Functions in Jump Table: Explanation

- Initially, all 8259A interrupts are masked out using mask on 8259A.
- ***startup*** and ***shutdown*** functions:
 - startup is called when first handler is installed for an interrupt.
 - shutdown is called after last handler is removed for an interrupt.
 - both functions change the corresponding mask bit in 8259A implementation.

Startup and Shutdown

- Startup and shutdown call enable / disable.

```
static unsigned int startup_8259A_irq(unsigned int irq)
{
    enable_8259A_irq(irq);
    return 0; /* never anything pending */
}
```



```
void enable_8259A_irq(unsigned int irq)
{
    unsigned int mask = ~(1 << irq);
    unsigned long flags;

    spin_lock_irqsave(&i8259A_lock, flags);
    cached_irq_mask &= mask;
    if (irq & 8)
        outb(cached_A1, 0xA1);
    else
        outb(cached_21, 0x21);
    spin_unlock_irqrestore(&i8259A_lock, flags);
}
```

Shutdown and Disable

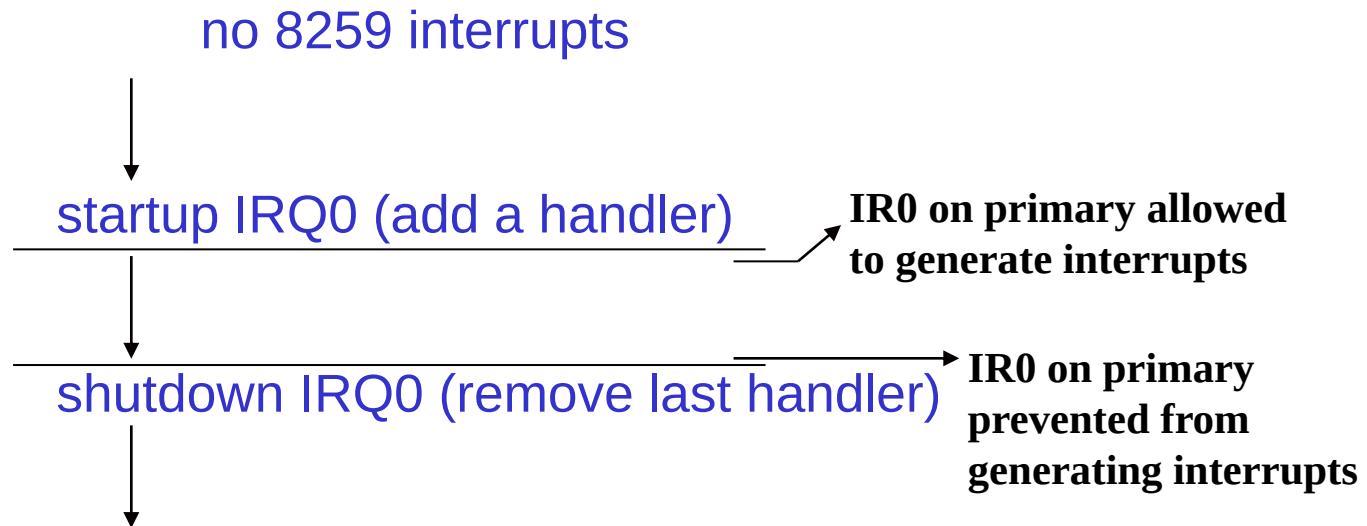
- [i8259.c - arch/i386/kernel/i8259.c - Linux source code \(2.4.28\) - Bootlin](#)

```
void disable_8259A_irq(unsigned int irq)
{
    unsigned int mask = 1 << irq;
    unsigned long flags;

    spin_lock_irqsave(&i8259A_lock, flags);
    cached_irq_mask |= mask;
    if (irq & 8)
        outb(cached_A1, 0xA1);
    else
        outb(cached_21, 0x21);
    spin_unlock_irqrestore(&i8259A_lock, flags);
}
```

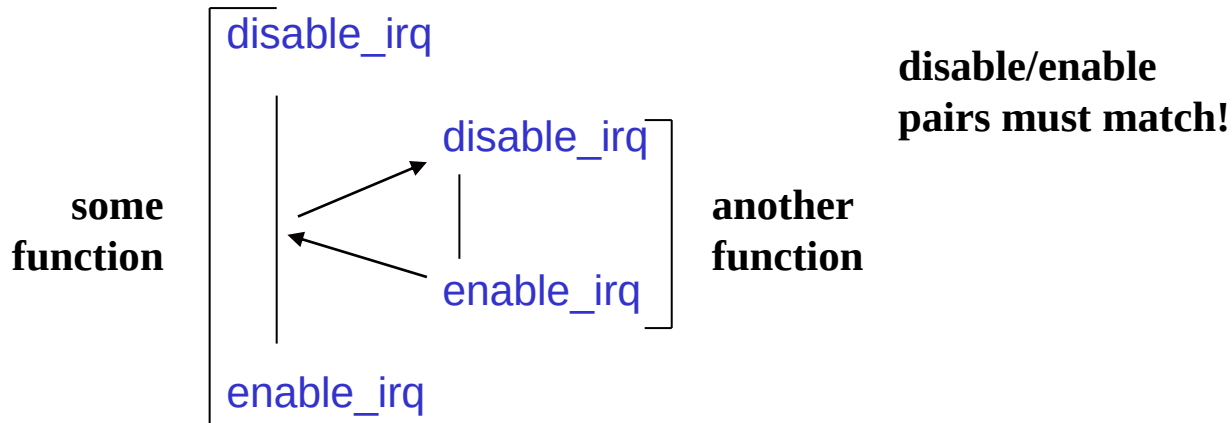
PIC Functions in Jump Table: Explanation (cont.)

- Example:



PIC Functions in Jump Table (cont.)

- disable/enable functions:
 - used to support nestable interrupt masking (`disable_irq`, `enable_irq`).



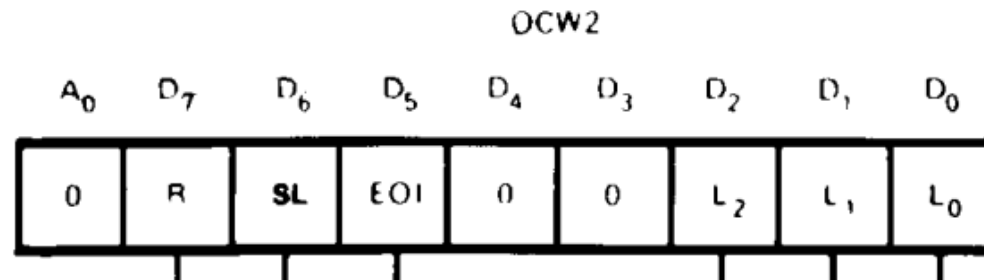
- on 8259:
 - first *disable_irq* calls jump table disable, which masks interrupt on PIC.
 - last *enable_irq* calls jump table enable, which unmaskes interrupt on PIC.

PIC Functions in Jump Table (cont.)

- **ack function:**
 - called at start of interrupt handling to ack receipt of the interrupt.
 - on 8259 (mask and ack), masks interrupt on PIC, then sends EOI to PIC.
- **end function:**
 - called at end of interrupt handling.
 - on 8259, enables interrupt (unmasks it) on PIC.

Mask and Ack

- Recall: in cascade mode, you need to send EOIs to both master and slave for slave interrupts.



```
handle_real_irq:
    if (irq & 8) {
        inb(0xA1);                /* DUMMY - (do we need this?) */
        outb(cached_A1, 0xA1);
        outb(0x60+(irq&7), 0xA0); /* 'Specific EOI' to slave */
        outb(0x62, 0x20);         /* 'Specific EOI' to master-IRQ2 */
    } else {
        inb(0x21);                /* DUMMY - (do we need this?) */
        outb(cached_21, 0x21);
        outb(0x60+irq, 0x20);     /* 'Specific EOI' to master */
    }
    spin_unlock_irqrestore(&i8259A_lock, flags);
    return;
```

General Interrupt Abstractions: Interrupt Chaining

- Hardware view: 1 interrupt $\rightarrow$ 1 handler.
- Problems:
 - may have > 15 devices.
 - > 1 software routines may want to act in response to device.
 - examples:
 - hotkeys for various functions.
 - move mouse to lower-right corner to start screen-saver.